

IFN647 Week 5 Workshop Tasks

Solutions

Task 1

Solution:

```
import glob, os
import string
from stemming.porter2 import stem

def index_docs(inputpath, stop_words):
    Index = {} # initialize the index
    os.chdir(inputpath)
    for file_ in glob.glob("*.xml"):
        start_end = False
        for line in open(file_):
            line = line.strip()
            if (start_end == False):
                if line.startswith("<newsitem "):
                    for part in line.split():
                        if part.startswith("itemid="):
                            docid = part.split("=")[1].split("\")[1]
                            break
                if line.startswith("<text>"):
                    start_end = True
            elif line.startswith("</text>"):
                break
            else:
                line = line.replace("<p>", "").replace("</p>", "")
                line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
                for term in line.split():
                    term = stem(term.lower())
                    if len(term) > 2 and term not in stop_words:
                        try:
                            try:
                                Index[term][docid] += 1
                            except KeyError:
                                Index[term][docid] = 1
                        except KeyError:
                            Index[term] = {docid:1}
    return Index
```

Task 2

Solution:

```
def doc_at_a_time(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={} # L is the selected inverted list
    R={} # R is a dictionary of docId:relevance
    for list in I.items():
        for id in list[1].items(): # get all document IDs with value 0
            R[id[0]]=0
        if (list[0] in Q): # select inverted lists based on the query
            L[list[0]] = I[list[0]]
    for (d, sd) in R.items():
        for (term, f) in L.items():
            if (d in f):
                sd = sd + f[d]*Q[term]
    R[d] = sd
    return R
```

Task 3

Solution:

```
def term_at_a_time(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={} # L is the selected inverted list
    R={} # R is a dictionary of docId:relevance
    for list in I.items():
        for id in list[1].items(): # get all document IDs with value 0
            R[id[0]]=0
        if (list[0] in Q): # select inverted lists based on the query
            L[list[0]] = I[list[0]]
    for (term, li) in L.items(): # traversal of the selected inverted list
        for (d, f) in li.items(): # for each occurrence of doc, update R
            R[d] = R[d] + f*Q[term]
    return R
```

Task 4

Solution:

```
import sys

curr_path=os.getcwd()
print(curr_path)

stopwords_f = open('common-english-words.txt', 'r')
stop_words = stopwords_f.read().split(',')
stopwords_f.close()
#Index = index_docs(sys.argv[1], stop_words) #create an index for all terms in <text>, data structure {'w1':{'ID1':2, 'ID2':1}, 'w2':
{'ID3':1, 'ID1':3}}
"""    for term in coll.items():
        print "Term --- %s" % (term[0])
        for id in coll[term[0]].items():
            print "    Document ID: %s and frequency: %d" % (id[0], id[1]) """
#Query = {'leaderboard':1, 'british':1}
#print(Index)

data_path = curr_path+'/data'
Index = index_docs(data_path, stop_words) #create an index for all terms in <text>, data structure {'w1':{'ID1':2, 'ID2':1}, 'w2':
{'ID3':1, 'ID1':3}}
os.chdir(curr_path)
print(Index)

Query = {'formula':1, 'one':1}
result1 = doc_at_a_time(Index, Query)
result2 = term_at_a_time(Index, Query)
x1 = sorted(result1.items(), key=lambda x: x[1],reverse=True)
x2 = sorted(result2.items(), key=lambda x: x[1],reverse=True)
print('Document_at_a_time result-----')
for (id, w) in x1:
    if w>0:
        print('Document ID: '+id + ' and relevance weight: ' + str(w))
print('Term_at_a_time result -----')
for (id, w) in x2:
    if w>0:
        print('Document ID: ' + id + ' and relevance weight: ' + str(w))
```